LAB 1

CUSTOM COMPUTER VISION OBJECT DETECTION WITH YOLO26

Course	DDM501
Weight	5%
Format	Team Lab (3-4 members per team)

1. OVERVIEW

1.1. Introduction

The purpose of this lab is to build a high-performance computer vision edge-product: a distributed **Automated Parking Car Detection System** utilizing a microservices architecture.

In real-world Intelligent Transportation Systems (ITS) and smart cities, deploying vision models on low-power edge nodes requires decoupling I/O-bound stream decoding from compute-bound neural network execution. This lab simulates an industrial-grade engineering pipeline:

- Structuring and labeling custom spatial data in YOLO format.
- Fine-tuning a state-of-the-art, edge-optimized object detector (**Ultralytics YOLO26**).
- Building an I/O-efficient **StreamIngestion** service to handle RTSP IP camera feeds.
- Implementing a high-throughput **ObjectDetectionInference** microservice wrapped in a low-latency **gRPC** server.
- Containerizing the multi-service network with Docker and writing robust integration tests using pytest-grpc.

1.2. Scenario: Automated Parking Car Detection System

You are a Principal CV Deployment Engineer at an urban mobility software provider. Your client, a automated parking garage operator, has high-resolution IP cameras feeding raw RTSP video streams from garage entryways and parking structures.

The local gateway devices run on low-power ARM-based edge nodes without discrete GPUs. To maintain real-time performance and optimal resource allocation, you must separate the architecture into two microservices:

1. **StreamIngestion Service**: Consumes high-rate frames directly from the IP camera stream, decodes, downsamples, and formats the images, then dispatches them to the inference pipeline before handling the parsed spatial boundaries.
2. **ObjectDetectionInference Service**: A dedicated service hosting a fine-tuned, ultra-lightweight **YOLO26-Nano** model. This service runs a gRPC server that takes raw frame payloads, conducts inference using YOLO26's native end-to-end NMS-free pipeline, and returns predicted class names and bounding box boundaries.

2. BACKGROUND

2.1. Object Detection & Evolution of YOLO26

Object detection is a core computer vision task that answers two questions simultaneously: *What objects are in the image, and where are they located?* This is solved by outputting class probabilities and bounding box coordinates for each detected item.

YOLO (You Only Look Once) revolutionized object detection by reframing it as a single regression problem, passing the image through the network just once to predict bounding boxes and classes.

YOLO26, released by Ultralytics, is a highly optimized, deployment-ready architecture designed specifically for high-efficiency edge-device execution (yielding up to 43% faster CPU inference than YOLO11).

2.2. YOLO Spatial Annotation Format

To train an object detector, bounding boxes must be formatted in normalized coordinates. For each object in an image, a corresponding line in a .txt label file contains:

$$\langle \text{class\_id} \rangle \quad \langle x_{\text{center}} \rangle \quad \langle y_{\text{center}} \rangle \quad \langle \text{width} \rangle \quad \langle \text{height} \rangle$$

Where:

- class_id : Integer representing the object class (e.g., 0 for car, 1 for motorcycle, 2 for truck).
- Normalized values (ranging from 0.0 to 1.0 relative to the image width and height):

$$x_{\text{center}} = \frac{X_{\text{box\_center}}}{W_{\text{image}}}, \quad y_{\text{center}} = \frac{Y_{\text{box\_center}}}{H_{\text{image}}}$$

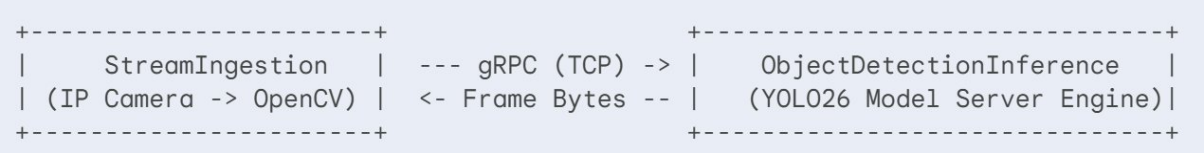
$$\text{width} = \frac{\text{Width}_{\text{box}}}{W_{\text{image}}}, \quad \text{height} = \frac{\text{Height}_{\text{box}}}{H_{\text{image}}}$$

2.3. Distributed gRPC vs. Monolithic REST APIs for Computer Vision

In video analytics, transport overhead is a major system bottleneck. Traditional HTTP/1.1 REST APIs require parsing heavy multipart/form-data MIME headers and encoding/decoding images in costly base64 strings or complex HTTP payloads. Furthermore, the persistent head-of-line blocking in HTTP/1.1 slows down real-time frame evaluation.

gRPC (Google Remote Procedure Call) resolves these problems by using **HTTP/2** as its transport layer and **Protocol Buffers (Protobuf)** as its interface definition language. It improves edge performance through several key features:

- **Binary Serialization:** Protobuf packs structured data into highly compressed binary payloads instead of heavy text-based JSON, minimizing network packet sizes.
- **Multiplexing over HTTP/2:** Multiple requests and responses can be interleaved over a single TCP connection, eliminating the overhead of frequent handshakes.
- **Stream-Ready Architecture:** Naturally supports client-streaming, server-streaming, and bidirectional streaming. This is ideal for continuous IP camera frame processing.



3. HANDS-ON GUIDES

Task 1: Setup Development Environment

1. **Create Project Structure:** Set up a clean, multi-module workspace supporting

independent development of the ingestion and inference microservices.
2. **Setup Virtual Environment:** Maintain dependencies under Python 3.10+.
3. **Install gRPC Tools:** Set up the compiler tools (grpcio and grpcio-tools) along with ultralytics and basic CV libraries.

Task 2: Dataset Preparation & Configuration

1. **Curate Parking Lot Dataset:** Group labeled parking lot images into training and validation folders.
2. **Formulate YOLO Configuration:** Write the dataset.yaml metadata file to train the model on local classes.

Task 3: Custom Model Fine-Tuning

1. **Train YOLO26-Nano:** Run the train_yolo26.py script on your custom dataset to generate optimized model weights.
2. **Evaluate Performance:** Verify that spatial metrics (like mAP_{50}) meet target operational standards.
3. **Save Model Weights:** Ensure the compiled best.pt is exported to the model assets directory.

Task 4: Develop gRPC Microservices

1. **Define Protocol Buffer Contract:** Author protos/detector.proto containing request schemas (raw frame bytes) and responses (bounding box coordinates, names, confidence ratings).
2. **Compile Protocol Buffers:** Use grpcio-tools to compile the interface into Python modules.
3. **Implement Inference Service (inference_service):** Write a standalone gRPC server that loads the custom YOLO26 weights, listens on port 50051, and exposes an endpoint for inference.
4. **Implement Stream Ingestion Service (stream_ingestion):** Write an ingestion client that connects to the IP camera stream, handles image resizing, and streams frame buffers to the inference server.

Task 5: Containerize and Test the Pipeline

1. **Dockerize Microservices:** Implement a multi-target Dockerfile that builds and configures either the ingestion client or the inference server.
2. **Compose Microservices Setup:** Configure docker-compose.yml to launch both containers in an isolated backend bridge network.
3. **Develop Integration Tests:** Build a test suite (tests/test_grpc.py) to verify the gRPC pipeline with simulated inputs.

4. INSTRUCTION STEPS

a) Project Directory Layout

a. Set up the following workspace structure:

b. parking-detector/
```
├── protos/
│   └── detector.proto          # Protocol Buffers service definition
├── stream_ingestion/
```

```
│   ├── __init__.py
│   └── ingest.py              # IP camera frame reader & gRPC client
├── inference_service/
│   ├── __init__.py
│   ├── server.py              # gRPC Object Detection server
│   └── detector.py            # YOLO26 loading & inference wrapper
├── data/
│   ├── dataset.yaml           # YOLO training configurations
│   ├── train/
│   │   ├── images/
│   │   └── labels/
│   └── val/
│       ├── images/
│       └── labels/
├── tests/
│   ├── __init__.py
│   └── test_grpc.py           # Automated gRPC test suite
├── train_yolo26.py            # YOLO26 training script
├── Dockerfile                 # Multi-target container specification
├── docker-compose.yml         # Multi-service local orchestrator
├── requirements.txt           # System dependencies
└── README.md                  # Deployment documentation
```

b) Dependency Configuration (requirements.txt)

c) Dataset Configuration (data/dataset.yaml)

d) Training Script (train_yolo26.py)

e) Protocol Buffer Interface Definition (protos/detector.proto)

f) Inference Core Engine (inference_service/detector.py)

g) Inference gRPC Server (inference_service/server.py)

h) Stream Ingestion Pipeline (stream_ingestion/ingest.py)

i) Containerization (Dockerfile)

j) Orchestration (docker-compose.yml)

k) Integration Tests (tests/test_grpc.py)

5. DELIVERABLES & GRADING

5.1. Deliverables

Teams must submit a comprehensive source directory (or Github Repository link) containing:

1. **Working Model & Training Pipeline:** The fine-tuned weights (best.pt) along with evaluation training runs in runs/train.
2. **Compiled Protocol Buffers Contract:** The interface configuration (protos/detector.proto) along with its compiled python files.
3. **StreamIngestion Module:** A resilient ingestion service capable of connecting to IP

cameras and decoding frame sequences.

4. **ObjectDetectionInference Microservice:** Standalone, high-throughput gRPC service wrapping the model.
5. **Docker Compose Orchestration:** Complete Dockerfile and docker-compose.yml that run both services in an isolated backend network.
6. **Automated Test Suite:** Passing tests verifying both happy paths and edge cases (empty payloads, connection issues).

5.2. Grading Rubric

Criteria	Weight	Detailed Description
Working ML Model	**25%**	Model loads successfully via Ultralytics API (10%), valid detections returning correct custom object classes (10%), fine-tuned mAP evaluation curves logged properly (5%).
gRPC Inference Server	**25%**	Correct implementation of the .proto schema (5%), fully operational inference service handling binary requests (10%), error handling for invalid input sizes or corrupted bytes (5%), proper gRPC status codes (5%).
Stream Ingestion Module	**20%**	Successful stream decoding and downscaling (8%), low-latency image compression (e.g., JPEG conversion) (7%), reconnection fallback logic (5%).
Docker & gRPC Orchestration	**15%**	Dockerfile successfully builds base and target layers (5%), multi-container network starts clean (5%), integration testing passing over mock gRPC channel (5%).
Documentation & Quality	**15%**	Complete README instructions with a schema diagram (5%), gRPC setup

		guide (5%), code complies with PEP8 and contains docstrings and typings (5%).

5.3. Submission Instructions

- **Deadline:** 1 week following the lab session.
- **Submission Format:** Source code package in standard .zip format or private GitHub repository link with viewer access granted to your course instructor.